

# Entwicklung eines Frameworks zur Robotersimulation in Jupyterlab

Philipp Mikos

September 29, 2025

- 1 Einleitung
- 2 Anforderungen
- 3 Entwicklungsprozess
- 4 Aufbau
- 5 Evaluation

- Entwicklung eines Frameworks zur Robotersimulation in Jupyterlab
- Basiert auf einer Projektarbeit
- Erweiterung des Frameworks um Simulation eines Industrieroboter
- Hosting des Frameworks auf einem Server im IDIAL
- Ziel: Einsatz in der Lehre
- Hilfe um individuell anpassbare Übungsszenarien zu erstellen

## **Funktionale Anforderungen:**

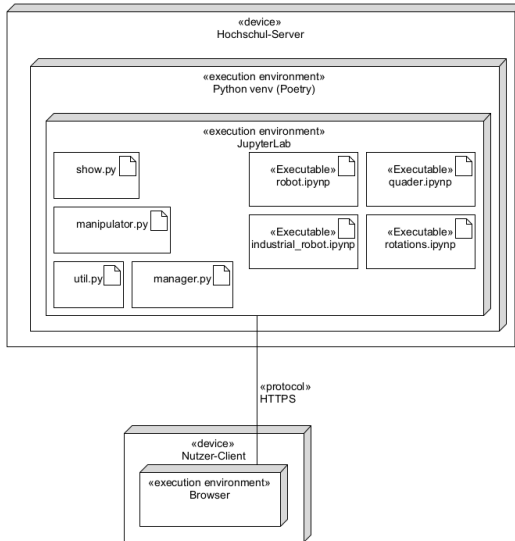
- Integration eines Industrieroboters mit Knickarmstruktur
- Umsetzung der Vorwärts- und Rückwärtstransformation
- Endeffektorsteuerung durch interaktive Bewegung des Koordinatensystems
- Bewegung im lokalen und globalen Koordinatensystem
- Integration einer Teach-In Funktion
- Manuelle Konfiguration der Gelenkwinkel

## **Nicht-Funktionale Anforderungen:**

- 60 FPS und Aktualisierungsrate der Animation von 60 Hz unter typischen Systemvoraussetzungen
- Die GUI soll intuitiv und ohne Schulung bedienbar sein, Hauptfunktionen mit maximal drei Klicks erreichbar sein, visuelles Feedback bei Fehlbedienung
- Die GUI soll so gestaltet sein, dass die Bedienung des Roboters gleichzeitig als Lernprozess dient

- Beschaffung der 3D-Modelle von Industrierobotern
- Geeignete Quellen: ROS-Industrial und die Robotics Toolbox von Peter Corke
- 3D-Modelle performant über Framework in den Arbeitsspeicher laden mithilfe des NPZ-Formats
- Rendering mit pythreejs mithilfe der Klasse "Environment" → Ausgabe der 3D-Szene mit 3D-Modellen direkt im Jupyter Notebook

- URDF-Beschreibungen laden und parsen
- Informationen aus URDF-Beschreibungen wichtig um 3D-Modelle richtig zu animieren
- Algorithmus zum Animieren: Zeitvariable  $0 \leq t$  solange um ein  $\Delta t$  erhöhen, wie  $t \leq 1$  ist
- In jeder Iteration wird Rotation mehrerer Gelenke um Drehachse interpoliert
- Rückwärtstransformation umgesetzt auf Basis der Newton-Raphson-Methode um Roboter mit mehr als 6 Gelenken zu unterstützen



## Weitere Abhängigkeiten:

- Numpy
- SymPy
- SciPy
- pythreejs(3D Rendering)
- ipywidgets(GUI-Elemente)



- Integration eines Knickarmroboters wurde umgesetzt
- Vorwärts- und Rückwärtstransformation ist implementiert worden
- Endeffektorsteuerung durch interaktive Bewegung des Koordinatensystems wurde umgesetzt
- Bewegung im lokalen und globalen Koordinatensystem möglich
- Teach-In Funktion wurde integriert
- Manuelle Konfiguration der Gelenkwinkel ist mithilfe von Schiebereglern umgesetzt worden

- Benutzerfreundliche Bedienbarkeit → müsste eine Untersuchung durch Experten oder eine Nutzerstudie erfolgen
- Intuitive Vermittlung kinematischer Konzepte → müsste eine Untersuchung durch Experten oder eine Nutzerstudie erfolgen
- Performante Ausführung (60FPS und 60Hz bei Aktualisierungsrate der Animation) konnte erfüllt werden

**Vielen Dank für Ihre  
Aufmerksamkeit!**